```
  }

  async updateData(): Promise<void> {
    const response = await fetch("/sold_out_concerts")
    const jsonString = await response.text()
    const jsonObject = JSON.parse(jsonString)
    const soldOutConcertIds = jsonObject["sold_out_concert_ids"].map((id) =>
      id.toString()
    )
    this.concertTargets.forEach((concertElement: HTMLElement) => {
      concertElement.dataset.concertSoldOutValue =
        soldOutConcertIds.includes(concertElement.dataset.concertIdValue)
    })
  }
}
```

This is a regular Stimulus controller that, when it is connected the DOM, uses setInterval to call the updateData method every 60 seconds (1000 milliseconds x 60). The updateData method calls fetch, awaits a response, and then parses the JSON.

The Stimulus controller sets up an arbitrary number of targets named concert, and when it receives data, it loops through those targets using the concertTargets plural property generated by Stimulus. Inside the loop, it checks the data attributes of the target for a concertIdValue, which it compares to the list of sold-out IDs. It then sets a data attribute called concertSoldOutValue to true if the ID is in the JSON data, and false otherwise.

How does that help us? Looking at the fact that both the attributes start with concert and end in Value suggests that they are meant to be value data in a Stimulus ConcertController, which is in fact the case.

Let's take a look at how we set up the Stimulus controllers in our HTML. We want one version of the SoldOutDataController, so we declare that at the top of our schedule show page:

chapter_08/03/app/views/schedules/show.html.erb

```
<section data-controller="sold-out-data">
```

Then, in our concert display partial view, we declare each concert as a target of